

Helping Hand

Complete Architecture Documentation

Healthcare Marketplace Web Application

Django + HTMX + PostgreSQL + Tailwind CSS

Helping Hand is a healthcare marketplace that connects patients with certified healthcare staff (nurses, caregivers) through agencies. It features online booking with automated tiered pricing, role-based access control, gender-matched staff assignment, and patient review management.

Migrated from React/Firebase (NoSQL + localStorage) to a strict relational Django ORM backend with PostgreSQL.

Technology Stack

Backend:	Django >= 4.2, Python
Database:	PostgreSQL (psycopg2-binary)
Frontend:	Django Templates, Tailwind CSS (CDN), HTMX 2.0.4
Icons:	Lucide Icons (SVG)
Auth:	Django session-based (username + password)

Codebase at a Glance

- 4** Data Models — User, StaffProfile, Booking, Review
- 6** View Classes — Landing, Services, Support, Login, Register, Logout
- 4** Form Classes — Patient Reg, Agency Reg, Login, Profile Completion
- 3** Service Types — Nursing Care, Home Care, One-Time
- 6** Booking Statuses — Pending, Reviewing, Confirmed, In-Progress, Completed, Cancelled
- 3** Discount Tiers — Daily (0%), Weekly (10%), Monthly (25%)

Helping Hand — Complete Architecture Document

Helping Hand is a healthcare marketplace web application built with Django, HTMX, PostgreSQL, and Tailwind CSS. It connects patients with certified healthcare staff (nurses, caregivers) through agencies, enabling online booking, automated pricing, and review management.

Migrated from React/Firebase (NoSQL + localStorage) to strict relational Django ORM + PostgreSQL.

1. High-Level System Architecture

The application follows a classic Django MTV (Model-Template-View) architecture with server-side rendering enhanced by HTMX for dynamic interactions without full page reloads.

► MERMAID DIAGRAM

```
graph TB
    subgraph "Client Browser"
        A[" Web Browser"]
        B["HTMX Engine"]
        C["Lucide Icons"]
        D["Tailwind CSS"]
    end

    subgraph "Django Application Server"
        E[" URL Router<br/>(helpinghand/urls.py → core/urls.py)"]
        F[" Middleware Stack<br/>(Security, Session, CSRF, Auth)"]
        G[" Views Layer<br/>(Class-Based Views)"]
        H[" Forms Layer<br/>(Validation & Processing)"]
        I[" Models & ORM<br/>(Business Logic)"]
        J[" Templates<br/>(Django Template Engine)"]
        K[" Auth System<br/>(Session-Based)"]
        L[" Pricing Engine<br/>(pricing.py)"]
        M[" Admin Interface<br/>(/admin/)"]
    end

    subgraph "Data Layer"
        N[" PostgreSQL<br/>Database"]
        O[" Media Files<br/>(staff photos, discharge docs)"]
        P[" Static Files<br/>(CSS, JS)"]
    end

    A -->|"HTTP Request"| F
    B -->|"HTMX Partial Requests"| F
    F --> E
    E --> G
    G --> H
    G --> I
    G --> J
    G --> K
    I --> L
    I --> N
    J -->|"HTML Response"| A
    M --> I
    O -->|"Served in DEBUG"| A
    P -->|"Static Serving"| A

    style A fill:#3b82f6,color:#fff
    ... (truncated)
```

How the layers interact

Layer	Responsibility	Key Files
Client	Renders HTML, sends HTTP/HTMX requests	base.html, Tailwind, HTMX, Lucide
Middleware	Security, sessions, CSRF, authentication	settings.py MIDDLEWARE list
URL Router	Maps URLs to view classes	helpinghand/urls.py, core/urls.py
Views	Handles business logic, returns responses	core/views.py (6 view classes)
Forms	Validates user input, creates/updates models	core/forms.py (4 form classes)
Models/ORM	Data persistence, business rules	core/models.py (4 models)
Pricing	Computes tiered pricing automatically	core/pricing.py
Database	Stores all application data	PostgreSQL via psycopg2

I Note: The application was migrated from a React/Firebase frontend. Source references are documented in core/models.py lines 7–13.

2. Project Structure & Directory Layout

► MERMAID DIAGRAM

```

graph LR
    subgraph "Project Root: Helping Hand MAX"
        A["manage.py<br/>Django CLI entry point"]
        B[".env<br/>Environment variables"]
        C["requirements.txt<br/>Python dependencies"]

        subgraph "helpinghand/ – Project Config"
            D["settings.py<br/>Django configuration"]
            E["urls.py<br/>Root URL router"]
            F["wsgi.py / asgi.py<br/>Server entry points"]
            G["test_settings.py<br/>Test DB overrides"]
        end
    end

    subgraph "core/ – Main Application"
        H["models.py<br/>4 data models (525 lines)"]
        I["views.py<br/>6 view classes (343 lines)"]
        J["forms.py<br/>4 form classes (250 lines)"]
        K["urls.py<br/>6 URL patterns"]
        L["admin.py<br/>4 admin configs (163 lines)"]
        M["managers.py<br/>Custom QuerySets (110 lines)"]
        N["pricing.py<br/>Pricing engine (133 lines)"]
        O["validators.py<br/>Phone & PIN validators"]
        P["decorators.py<br/>Role-based FBV decorators"]
        Q["mixins.py<br/>Role-based CBV mixins"]
        R["tests/<br/>Test suite"]
    end

    subgraph "templates/"
        S["base.html<br/>Master layout"]
        T["core/<br/>5 templates"]
        U["auth/<br/>2 templates"]
    end

    subgraph "static/"
        V["css/style.css<br/>Design system"]
    end

    style D fill:#f59e0b,color:#000
    style H fill:#3b82f6,color:#fff
    style I fill:#10b981,color:#fff
    style N fill:#ef4444,color:#fff

```

File Inventory

File	Purpose	Size
core/models.py	User, StaffProfile, Booking, Review models	525 lines
core/views.py	Landing, Services, Support, Auth views	343 lines
core/forms.py	Registration, Login, Profile forms	250 lines
core/admin.py	Admin panel customization	163 lines
core/pricing.py	Service rates & discount calculations	133 lines
core/managers.py	Booking & StaffProfile custom querysets	110 lines
core/decorators.py	@patient_required, @agency_required, @profile_complete_required	70 lines
core/mixins.py	PatientRequiredMixin, AgencyRequiredMixin, ProfileCompleteRequired..	71 lines
core/validators.py	phone_validator (10 digits), pincode_validator (6 digits)	21 lines
helpinghand/settings.py	Django configuration (DB, Auth, etc.)	122 lines
templates/base.html	Master layout with Tailwind + HTMX + Lucide	114 lines

3. Technology Stack

► MERMAID DIAGRAM

```
graph TB
    subgraph "Frontend (CDN-Based)"
        A["Tailwind CSS<br/>(CDN, Play mode)"]
        B["HTMX 2.0.4<br/>(Dynamic partials)"]
        C["Lucide Icons<br/>(SVG icon library)"]
        D["Django Templates<br/>(Server-side rendering)"]
        E["Google Fonts<br/>(Inter, DM Sans)"]
    end

    subgraph "Backend Framework"
        F["Django ≥ 4.2, < 6.0<br/>(Python web framework)"]
        G["django-extensions ≥ 3.2<br/>(Dev utilities)"]
    end

    subgraph "Database"
        H["PostgreSQL<br/>(via psycopg2-binary ≥ 2.9)"]
    end

    subgraph "Utilities"
        I["python-dotenv ≥ 1.0<br/>(env file support)"]
        J["Pillow ≥ 10.0<br/>(Image handling for ImageField)"]
    end

    D --> F
    A --> D
    B --> D
    C --> D
    E --> D
    F --> H
    F --> I
    F --> J
    F --> G

    style F fill:#092e20,color:#fff
    style H fill:#336791,color:#fff
    style B fill:#3b82f6,color:#fff
```

Dependencies (requirements.txt)

PYTHON

```
# Core framework
Django>=4.2,<6.0

# PostgreSQL adapter
psycopg2-binary>=2.9

# Environment variables (.env support)
python-dotenv>=1.0

# Django developer utilities (shell_plus, runserver_plus, etc.)
django-extensions>=3.2

# Image handling (required by ImageField in StaffProfile)
Pillow>=10.0
```

4. Data Model & Entity-Relationship Diagram

The application uses 4 core models with a strict relational schema enforcing healthcare business rules at the database level.

► MERMAID DIAGRAM

```
erDiagram
    USER ||--o{ BOOKING : "creates (as patient)"
    USER ||--o{ STAFF_PROFILE : "manages (as agency)"
    STAFF_PROFILE ||--o{ BOOKING : "assigned to"
    BOOKING ||--o| REVIEW : "has one (1:1)"

    USER {
        BigAutoField id PK
        CharField username UK
        CharField role "patient | agency"
        CharField phone UK "10-digit, unique"
        CharField alternate_phone "optional"
        TextField address
        CharField address_line2
        CharField city
        CharField state
        CharField pincode "6-digit"
        BooleanField profile_complete
        DateTimeField date_joined
    }

    STAFF_PROFILE {
        BigAutoField id PK
        CharField name
        CharField gender "male | female"
        PositiveIntegerField age "min 18"
        ImageField photo "staff_photos/"
        JSONField specializations "list of strings"
        JSONField service_types "list: nursing, homecare, onetime"
        DecimalField hourly_rate "INR per day"
        DecimalField rating "0.0 to 5.0"
        PositiveIntegerField review_count
        BooleanField available "indexed"
        ForeignKey agency FK "→ User where role=agency"
        DateTimeField created_at
        DateTimeField updated_at
    }

    BOOKING {
        BigAutoField id PK
        CharField booking_id UK "BK-xxxxxxx (auto)"
        ForeignKey patient FK "→ User where role=patient"
    }
... (truncated)
```

Model Relationships Summary

► MERMAID DIAGRAM

```
graph LR
    U_Patient["User<br/>(role=patient)"] -->|"creates"| B["Booking"]
    U_Agency["User<br/>(role=agency)"] -->|"manages"| SP["StaffProfile"]
    SP -->|"assigned to"| B
    B -->|"has one"| R["Review"]

    style U_Patient fill:#3b82f6,color:#fff
    style U_Agency fill:#10b981,color:#fff
    style B fill:#f59e0b,color:#000
    style SP fill:#8b5cf6,color:#fff
    style R fill:#ef4444,color:#fff
```

User Model (core/models.py:35-115)

Extends Django's AbstractUser with two roles and healthcare-specific fields:

PYTHON

```
class User(AbstractUser):
    class Role(models.TextChoices):
        PATIENT = 'patient', 'Patient'
        AGENCY = 'agency', 'Agency Manager'

    role = models.CharField(
        max_length=10,
        choices=Role.choices,
        default=Role.PATIENT,
        db_index=True, # Optimized for role-based queries
    )
    phone = models.CharField(max_length=15, unique=True, validators=[phone_validator])
    profile_complete = models.BooleanField(default=False)

    @property
    def is_patient(self):
        return self.role == self.Role.PATIENT

    @property
    def is_agency(self):
        return self.role == self.Role.AGENCY
```

■ Important: `AUTH_USER_MODEL = 'core.User'` in `settings.py:90` replaces Django's default User model.

StaffProfile Model (core/models.py:121-219)

Healthcare staff managed by agencies, with JSONField for flexible service types:

PYTHON

```

class StaffProfile(models.Model):
    class Gender(models.TextChoices):
        MALE = 'male', 'Male'
        FEMALE = 'female', 'Female'

    class ServiceType(models.TextChoices):
        NURSING = 'nursing', 'Nursing Care'
        HOMECARE = 'homecare', 'Home Care'
        ONETIME = 'onetime', 'One-Time Service'

    name = models.CharField(max_length=200)
    gender = models.CharField(max_length=10, choices=Gender.choices)
    age = models.PositiveIntegerField(validators=[MinValueValidator(18)])
    specializations = models.JSONField(default=list) # ["Nursing Care", "Post-Surgery"]
    service_types = models.JSONField(default=list) # ["nursing", "onetime"]
    hourly_rate = models.DecimalField(max_digits=8, decimal_places=2)
    rating = models.DecimalField(max_digits=3, decimal_places=1, default=Decimal('0.0'))
    available = models.BooleanField(default=True, db_index=True)
    agency = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name='staff_profiles',
        limit_choices_to={'role': User.Role.AGENCY},
    )

```

Booking Model — Business Rules (core/models.py:225-480)

The central transaction model enforcing healthcare rules:

PYTHON

```

class Booking(models.Model):
    class Status(models.TextChoices):
        PENDING = 'pending', 'Pending'
        REVIEWING = 'reviewing', 'Under Review'
        CONFIRMED = 'confirmed', 'Confirmed'
        IN_PROGRESS = 'in-progress', 'In Progress'
        COMPLETED = 'completed', 'Completed'
        CANCELLED = 'cancelled', 'Cancelled'

    # Auto-generated unique ID
    booking_id = models.CharField(max_length=20, unique=True, editable=False)

    # Relationships with role constraints
    patient = models.ForeignKey(User, limit_choices_to={'role': 'patient'}, ...)
    staff = models.ForeignKey(StaffProfile, null=True, blank=True, ...)

    # Pricing fields (auto-computed on every save())
    base_rate = models.DecimalField(...)
    discount_percentage = models.DecimalField(...)
    total_cost = models.DecimalField(...)

    def clean(self):
        """Enforce: gender matching + service type compatibility"""
        if self.staff and self.staff.gender != self.patient_gender:
            raise ValidationError('Staff gender must match patient gender.')
        if self.staff and self.service_type not in self.staff.service_types:
            raise ValidationError('Staff does not provide this service.')

    def save(self, *args, **kwargs):
        if not self.booking_id:
            self.booking_id = f'BK-{uuid.uuid4().hex[:8]}'
        self._compute_pricing() # Always recompute
        if not self.pk:
            self._determine_initial_status()
        super().save(*args, **kwargs)

```


Review Model (core/models.py:486-525)

One review per booking (1:1), only for completed bookings:

PYTHON

```
class Review(models.Model):
    booking = models.OneToOneField(
        Booking,
        on_delete=models.CASCADE,
        limit_choices_to={'status': 'completed'},
    )
    rating = models.PositiveIntegerField(
        validators=[MinValueValidator(1), MaxValueValidator(5)]
    )
    comment = models.TextField(blank=True, default='')

    def clean(self):
        if self.booking.status != Booking.Status.COMPLETED:
            raise ValidationError('Reviews only for completed bookings.')
```

5. Django Request-Response Lifecycle

This diagram traces a complete HTTP request through every layer:

► MERMAID DIAGRAM

```
sequenceDiagram
    participant Browser
    participant Middleware as Middleware Stack
    participant URLRouter as URL Router
    participant View as View (CBV)
    participant Form as Form
    participant Model as Model/ORM
    participant DB as PostgreSQL
    participant Template as Template Engine

    Browser->>Middleware: HTTP Request (GET/POST)

    Note over Middleware: 1. SecurityMiddleware (HTTPS, HSTS)
    Note over Middleware: 2. SessionMiddleware (session data)
    Note over Middleware: 3. CommonMiddleware (URL normalization)
    Note over Middleware: 4. CsrfViewMiddleware (CSRF token)
    Note over Middleware: 5. AuthenticationMiddleware (request.user)
    Note over Middleware: 6. MessageMiddleware (flash messages)
    Note over Middleware: 7. XFrameOptionsMiddleware (clickjacking)

    Middleware->>URLRouter: Processed Request
    URLRouter->>URLRouter: Match URL pattern
    URLRouter->>View: Dispatch to View

    alt GET Request (e.g., Landing Page)
        View->>Model: Query data (ORM)
        Model->>DB: SQL SELECT
        DB-->>Model: Result set
        Model-->>View: QuerySet / Objects
        View->>Template: Render with context dict
        Template-->>View: HTML string
    else POST Request (e.g., Registration)
        View->>Form: Bind POST data
        Form->>Form: field-level validation
        Form->>Model: form.save()
        Model->>Model: clean() – business rules
        Model->>Model: save() – pricing, IDs
        Model->>DB: SQL INSERT/UPDATE
        DB-->>Model: Saved instance
        Model-->>View: Success
        View->>View: messages.success()
        View->>View: redirect()
    end
    ... (truncated)
```

Middleware Stack (settings.py:43-51)

PYTHON

```
MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',          # HTTPS, HSTS headers
    'django.contrib.sessions.middleware.SessionMiddleware',   # Session cookies
    'django.middleware.common.CommonMiddleware',             # URL normalization
    'django.middleware.csrf.CsrfViewMiddleware',            # CSRF protection
    'django.contrib.auth.middleware.AuthenticationMiddleware', # Sets request.user
    'django.contrib.messages.middleware.MessageMiddleware',   # Flash messages
    'django.middleware.clickjacking.XFrameOptionsMiddleware', # X-Frame-Options
]
```

6. URL Routing Architecture

The application uses a two-level URL routing strategy:

► MERMAID DIAGRAM

```
graph TD
    A["Incoming Request URL"] --> B["helpinghand/urls.py<br/>(Root Router)"]

    B -->|"/admin/*"| C[" Django Admin<br/>admin.site.urls"]
    B -->|"/*"| D["core/urls.py<br/>(App Router)"]

    D -->|"GET /"| E[" LandingView<br/>name='landing'"]
    D -->|"GET /services/"| F[" ServicesView<br/>name='services'"]
    D -->|"GET /support/"| G[" SupportView<br/>name='support'"]
    D -->|"GET|POST /auth/login/"| H[" CustomLoginView<br/>name='login'"]
    D -->|"GET|POST /auth/register/"| I[" RegisterView<br/>name='register'"]
    D -->|"POST /auth/logout/"| J[" CustomLogoutView<br/>name='logout'"]

    B -->|"/media/* (DEBUG only)"| K[" Media Files<br/>static() serve"]

    style C fill:#8b5cf6,color:#fff
    style E fill:#3b82f6,color:#fff
    style F fill:#3b82f6,color:#fff
    style G fill:#3b82f6,color:#fff
    style H fill:#ef4444,color:#fff
    style I fill:#ef4444,color:#fff
    style J fill:#ef4444,color:#fff
```

Root URL Configuration (helpinghand/urls.py)

PYTHON

```
from django.contrib import admin
from django.urls import path, include
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    path('admin/', admin.site.urls),      # Django admin panel
    path('', include('core.urls')),      # All app routes delegated to core
]

# Serve media files in development only
if settings.DEBUG:
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.MEDIA_ROOT)
```

App URL Configuration (core/urls.py)

PYTHON

```
from django.urls import path
from . import views

urlpatterns = [
    # Public Pages
    path('', views.LandingView.as_view(), name='landing'),
    path('services/', views.ServicesView.as_view(), name='services'),
    path('support/', views.SupportView.as_view(), name='support'),

    # Authentication
    path('auth/login/', views.CustomLoginView.as_view(), name='login'),
    path('auth/register/', views.RegisterView.as_view(), name='register'),
    path('auth/logout/', views.CustomLogoutView.as_view(), name='logout'),
]
```

URL → View → Template Mapping

URL Pattern	View Class	Template	Description
/	LandingView	core/landing.html	Hero, services, how-it-works, pricing, testim..
/services/	ServicesView	core/services.html	Detailed service cards with features
/support/	SupportView	core/support.html	Searchable FAQ accordion + contact
/auth/login/	CustomLoginView	auth/login.html	Username + password login
/auth/register/	RegisterView	auth/register.html	Role-based registration
/auth/logout/	CustomLogoutView	—	Logout + redirect to landing
/admin/	Django Admin	Built-in	Full admin panel

7. Authentication & Authorization System

Registration Flow

► MERMAID DIAGRAM

```

flowchart TD
    A["User visits<br>/auth/register/"] --> B["Already<br>authenticated?"]
    B -->|"Yes"| C["Redirect to Landing"]
    B -->|"No"| D["?role= query param"]

    D -->|"role=patient<br>(or missing)"| E["PatientRegistrationForm"]
    D -->|"role=agency"| F["AgencyRegistrationForm"]

    E --> G["Fill: username, first_name,<br>last_name, phone, email,<br>password1, password2"]
    F --> H["Fill: username, first_name,<br>last_name, phone,<br>password1, password2"]

    G --> I["form.is_valid()?"]
    H --> I

    I -->|"No"| J["Re-render with<br>validation errors"]
    J --> G
    J --> H

    I -->|"Yes"| K["form.save()"]
    K --> L["Auto-set role:<br>PATIENT or AGENCY"]
    L --> M["login(request, user)<br>Auto-login"]
    M --> N["messages.success(<br>'Welcome to Helping Hand!')"]
    N --> O["Redirect to<br>Landing Page (/)"]

    style E fill:#3b82f6,color:#fff
    style F fill:#10b981,color:#fff
    style M fill:#f59e0b,color:#000

```

Registration Code (core/views.py:266-308)

PYTHON

```

class RegisterView(FormView):
    template_name = 'auth/register.html'
    success_url = reverse_lazy('landing')

    def get_role(self):
        """Determine role from ?role= query parameter."""
        role = self.request.GET.get('role', 'patient')
        return role if role in ('patient', 'agency') else 'patient'

    def get_form_class(self):
        """Return the correct form based on selected role."""
        if self.get_role() == 'agency':
            return AgencyRegistrationForm
        return PatientRegistrationForm

    def form_valid(self, form):
        user = form.save()          # Creates user with auto-set role
        login(self.request, user)   # Auto-login after registration
        messages.success(self.request, f'Welcome to Helping Hand, {user.get_full_name()}!')
        return redirect(self.success_url)

    def dispatch(self, request, *args, **kwargs):
        if request.user.is_authenticated:
            return redirect('landing')
        return super().dispatch(request, *args, **kwargs)

```

Login/Logout Flow

► MERMAID DIAGRAM

```

flowchart LR
    subgraph "Login"
        L1["GET /auth/login/"] --> L2["Show HelpingHandLoginForm"]
        L2 --> L3["POST username + password"]
        L3 --> L4["Valid?"]
        L4 -->|"Yes"| L5["Redirect to<br/>LOGIN_REDIRECT_URL<br/>(/dashboard/)"]
        L4 -->|"No"| L6["Show error"]
    end

    subgraph "Logout"
        O1["POST /auth/logout/"] --> O2["Clear session"]
        O2 --> O3["messages.info('Logged out')"]
        O3 --> O4["Redirect to<br/>Landing Page (/)"]
    end

    style L5 fill:#10b981,color:#fff
    style O4 fill:#3b82f6,color:#fff

```

Auth Settings (settings.py:100-103)

PYTHON

```

LOGIN_URL            = '/auth/login/'      # Where @login_required redirects
LOGIN_REDIRECT_URL   = '/dashboard/'       # After successful login
LOGOUT_REDIRECT_URL  = '/'                 # After logout

```

Authorization — Dual Pattern (Decorators + Mixins)

The application provides two parallel authorization systems for maximum flexibility:

► MERMAID DIAGRAM

```

graph TB
    subgraph "Function-Based Views (decorators.py)"
        A["@patient_required"] --> A1["@login_required<br/>+ role == PATIENT<br/>else → 403"]
        B["@agency_required"] --> B1["@login_required<br/>+ role == AGENCY<br/>else → 403"]
        C["@profile_complete_required"] --> C1["@login_required<br/>+ profile_complete == True<br/>else → 403"]
    end

    subgraph "Class-Based Views (mixins.py)"
        D["PatientRequiredMixin"] --> D1["LoginRequiredMixin<br/>+ UserPassesTestMixin<br/>test: ..."]
        E["AgencyRequiredMixin"] --> E1["LoginRequiredMixin<br/>+ UserPassesTestMixin<br/>test: i..."]
        F["ProfileCompleteRequiredMixin"] --> F1["LoginRequiredMixin<br/>+ UserPassesTestMixin<br/>test: p..."]
    end

    style A fill:#ef4444,color:#fff
    style B fill:#ef4444,color:#fff
    style C fill:#ef4444,color:#fff
    style D fill:#3b82f6,color:#fff
    style E fill:#3b82f6,color:#fff
    style F fill:#3b82f6,color:#fff

```

Decorator usage example:

PYTHON

```
# For function-based views
@patient_required
def my_booking_view(request):
    # Only authenticated patients reach here
    bookings = Booking.objects.filter(patient=request.user)
    ...
```

Mixin usage example:

PYTHON

```
# For class-based views
class DashboardView(PatientRequiredMixin, TemplateView):
    template_name = 'core/dashboard.html'
    # Only authenticated patients reach here
```

Decorator implementation (core/decorators.py):

PYTHON

```
def patient_required(view_func):
    @wraps(view_func)
    @login_required
    def _wrapped(request, *args, **kwargs):
        if not request.user.is_patient:
            raise PermissionDenied('This page is only accessible to patient accounts.')
        return view_func(request, *args, **kwargs)
    return _wrapped
```

Mixin implementation (core/mixins.py):

PYTHON

```
class AgencyRequiredMixin(LoginRequiredMixin, UserPassesTestMixin):
    def test_func(self):
        return self.request.user.is_agency

    def handle_no_permission(self):
        if self.request.user.is_authenticated:
            raise PermissionDenied('Agency manager accounts only.')
        return super().handle_no_permission() # Redirect to login
```

8. Booking Lifecycle & State Machine

The Booking model implements a 6-state lifecycle with automated transitions:

► MERMAID DIAGRAM

```
stateDiagram-v2
    [*] --> Pending: New booking<br/>(no discharge file)
    [*] --> Reviewing: New booking<br/>(discharge file uploaded)

    Pending --> Confirmed: Agency accepts booking
    Reviewing --> Confirmed: Documents reviewed ✓
    Reviewing --> Pending: Documents rejected

    Confirmed --> InProgress: Service starts<br/>(on start_date)
    InProgress --> Completed: Service finished

    Pending --> Cancelled: Patient or Agency cancels
    Confirmed --> Cancelled: Cancelled before start

    Completed --> [*]: Review can be submitted (1:1)

    note right of Pending
        Default state for all new bookings
        that don't include a discharge file
    end note

    note right of Reviewing
        Auto-set by save() when discharge_file
        is uploaded during booking creation
    end note

    note right of Completed
        Only completed bookings can
        receive patient reviews (OneToOneField)
    end note
```

Status Values (core/models.py:239-246)

Status	DB Value	Display	Category
Pending	pending	Pending	Active
Under Review	reviewing	Under Review	Active
Confirmed	confirmed	Confirmed	Active
In Progress	in-progress	In Progress	Active
Completed	completed	Completed	Past
Cancelled	cancelled	Cancelled	Past

Automatic Status Determination (core/models.py:426-434)

PYTHON

```
def _determine_initial_status(self):
    """Set initial status based on whether a discharge file is uploaded."""
    if not self.pk or self.status == self.Status.PENDING:
        if self.discharge_file:
            self.status = self.Status.REVIEWING # Auto-escalate
        elif self.status not in [s[0] for s in self.Status.choices]:
            self.status = self.Status.PENDING
```


Booking ID Generation (core/models.py:406-409)

PYTHON

```
def _generate_booking_id(self):
    """Generate a unique booking ID with BK- prefix."""
    short_uuid = uuid.uuid4().hex[:8]
    return f'BK-{short_uuid}'
# Example output: BK-a1b2c3d4
```

Active vs. Past Bookings

PYTHON

```
@property
def is_active(self):
    return self.status in [
        self.Status.PENDING,
        self.Status.CONFIRMED,
        self.Status.IN_PROGRESS,
        self.Status.REVIEWING,
    ]
# Completed and Cancelled = "past" bookings

@property
def end_date(self):
    """Computed: start_date + duration_days - 1"""
    if self.start_date:
        return self.start_date + timedelta(days=self.duration_days - 1)
    return None
```

9. Pricing Engine

The pricing engine (core/pricing.py) implements a tiered discount system based on service type and booking duration.

► MERMAID DIAGRAM

```
graph TD
    A["calculate_price(service_type, duration_days)"] --> B["Service type<br/>valid?"]
    B -->| "No (not in SERVICE_RATES)" | C["Return None"]
    B -->| "Yes" | D["service_type ==<br/>'onetime'?"]
    D -->| "Yes" | E["Force 1 day<br/>No discount<br/>Return ₹500 flat"]
    D -->| "No" | F["How many<br/>days?"]
    F -->| "1-6 days" | G[" Tier: DAILY<br/>Discount: 0%"]
    F -->| "7-29 days" | H[" Tier: WEEKLY<br/>Discount: 10%"]
    F -->| "30+ days" | I[" Tier: MONTHLY<br/>Discount: 25%"]
    G --> J["subtotal = base_rate × days"]
    H --> J
    I --> J
    J --> K["discount_amount = subtotal × discount%<br/>(rounded half-up)"]
    K --> L["total = subtotal - discount_amount"]
    L --> M["Return complete<br/>pricing breakdown dict"]

    style A fill:#10b981,color:#fff
    style G fill:#3b82f6,color:#fff
    style H fill:#f59e0b,color:#000
    style I fill:#ef4444,color:#fff
```

Service Rates (INR per day)

Service Type	Daily Rate	Weekly Rate (10% off)	Monthly Rate (25% off)
Nursing Care	₹1,500/day	₹1,350/day	₹1,125/day
Home Care	₹1,000/day	₹900/day	₹750/day
One-Time Service	₹500 (flat)	N/A	N/A

Discount Tiers

Tier	Duration	Discount	Example (Nursing, 14 days)
Daily	1-6 days	0%	5 × ₹1,500 = ₹7,500
Weekly	7-29 days	10%	14 × ₹1,500 = ₹21,000 – ₹2,100 = ₹18,900
Monthly	30+ days	25%	30 × ₹1,500 = ₹45,000 – ₹11,250 = ₹33,750

Core Pricing Code (core/pricing.py)

PYTHON

```

SERVICE_RATES = {
    'nursing': {'daily': Decimal('1500'), 'label': 'Nursing Care'},
    'homecare': {'daily': Decimal('1000'), 'label': 'Home Care'},
    'onetime': {'daily': Decimal('500'), 'label': 'One-Time Service'},
}

DISCOUNT_TIERS = {
    'daily': {'min_days': 1, 'max_days': 6, 'discount': Decimal('0')},
    'weekly': {'min_days': 7, 'max_days': 29, 'discount': Decimal('0.10')},
    'monthly': {'min_days': 30, 'max_days': None, 'discount': Decimal('0.25')},
}

def calculate_price(service_type, duration_days):
    rate = SERVICE_RATES.get(service_type)
    if rate is None:
        return None

    if service_type == 'onetime':
        return {'base_rate': rate['daily'], 'total': rate['daily'],
                'tier': 'daily', 'discount': Decimal('0'), ...}

    tier = get_tier(duration_days)
    subtotal = rate['daily'] * duration_days
    discount_amount = (subtotal * tier['discount']).quantize(
        Decimal('1'), rounding=ROUND_HALF_UP
    )
    total = subtotal - discount_amount
    return {'base_rate': rate['daily'], 'subtotal': subtotal,
            'discount': tier['discount'], 'discount_amount': discount_amount,
            'total': total, 'tier': tier['label'], ...}

def format_currency(amount) -> str:
    """Format as Indian Rupees: format_currency(1500) -> '₹1,500'"""
    return f'₹{int(amount):,}'

```

Pricing is Auto-Computed in Booking.save()

PYTHON

```

def save(self, *args, **kwargs):
    if not self.booking_id:
        self.booking_id = self._generate_booking_id()
    self._compute_pricing()    # ← Always recompute on every save!
    if not self.pk:
        self._determine_initial_status()
    super().save(*args, **kwargs)

```

10. Custom Managers & QuerySets

The application implements the Manager-QuerySet pattern for chainable, reusable queries:

► MERMAID DIAGRAM

```
graph TD
    subgraph "BookingManager / BookingQuerySet"
        A["Booking.objects.active()"] --> A1["status ∈ {pending, confirmed,<br/>in-progress, revi..."]
        B["Booking.objects.past()"] --> B1["status ∈ {completed, cancelled}"]
        C["for_patient(user)"] --> C1["patient = user"]
        D["for_agency(user)"] --> D1["staff__agency = user"]
        E["by_service_type(type)"] --> E1["service_type = type"]
        F["needs_review()"] --> F1["status = 'reviewing'"]
    end

    subgraph "StaffProfileManager / StaffProfileQuerySet"
        G["StaffProfile.objects.available()"] --> G1["available = True"]
        H["by_gender(gender)"] --> H1["gender = gender"]
        I["by_service_type(type)"] --> I1["JSONField list containment<br/>(Python-level filterin..."]
        J["matching(gender, type)"] --> J1["available + gender match<br/>+ service type match<br..."]
    end

    style A fill:#3b82f6,color:#fff
    style J fill:#10b981,color:#fff
```

Chainable Query Examples

PYTHON

```
# Get all active nursing bookings for a patient
Booking.objects.active().for_patient(request.user).by_service_type('nursing')

# Get bookings awaiting discharge document review
Booking.objects.needs_review()

# Find available female staff who provide nursing care
StaffProfile.objects.matching('female', 'nursing')
```

Staff Matching Logic (core/managers.py:81-96)

Mirrors the original React findMatchingStaff() function:

PYTHON

```
def matching(self, patient_gender, service_type):
    """Find available staff matching both gender and service type."""
    base_qs = (
        self.available()
        .by_gender(patient_gender)
        .filter(age__gte=18)
    )
    # JSONField list containment (DB-agnostic approach)
    pks = [
        obj.pk for obj in base_qs
        if service_type in (obj.service_types or [])
    ]
    return base_qs.filter(pk__in=pks)
```

Note: `JSONField __contains` isn't supported on `SQLite`, so the code uses a Python-level filter that works on all database backends. On `PostgreSQL` in production, this can be optimized with native `__contains` lookups.

11. Forms & Validation Pipeline

The application uses a 4-layer validation strategy:

► MERMAID DIAGRAM

```
graph TD
    A["User submits form<br/>(POST request)"] --> B["Layer 1: Django Form Fields"]
    subgraph "Layer 1: Form Field Validation"
        B --> B1["Built-in: required, max_length,<br/>email format"]
        B1 --> B2["Widget attrs: CSS classes,<br/>placeholders, autocomplete"]
    end
    subgraph "Layer 2: Custom Validators (validators.py)"
        B2 --> C1["phone_validator<br/>RegexValidator: ^\d{10}$"]
        B2 --> C2["pincode_validator<br/>RegexValidator: ^\d{6}$"]
    end
    subgraph "Layer 3: Model clean() — Business Rules"
        C1 --> D1["Gender matching<br/>(staff.gender == patient_gender)"]
        C2 --> D2["Service type compatibility<br/>(service_type in staff.service_types)"]
        D1 --> D3["Any errors?"]
        D2 --> D3
        D3 --> D4["raise ValidationError"]
    end
    subgraph "Layer 4: Model save() — Auto-Processing"
        D4 --> E1["Auto-generate booking_id<br/>(BK-xxxxxxx)"]
        E1 --> E2["Compute pricing<br/>(tiered discounts)"]
        E2 --> E3["Determine initial status<br/>(pending or reviewing)"]
        E3 --> E4["super().save() → PostgreSQL"]
    end
```

style B fill:#3b82f6,color:#fff
style C1 fill:#f59e0b,color:#000
style D1 fill:#ef4444,color:#fff
style E1 fill:#10b981,color:#fff

Form Classes (core/forms.py)

Form Class	Parent	Purpose	Auto-sets
PatientRegistratio..	UserCreationForm	Patient sign-up (username, name, phone, emai..	role = PATIENT
AgencyRegistration..	UserCreationForm	Agency manager sign-up (username, name, phon..	role = AGENCY
HelpingHandLoginForm	AuthenticationF..	Styled login (username + password)	—
ProfileCompletionF..	ModelForm	Address & profile completion	profile_complete =..

Form save() — Role Assignment

PYTHON



```
# PatientRegistrationForm.save()
def save(self, commit=True):
    user = super().save(commit=False)
    user.role = User.Role.PATIENT # Always set to Patient
    if commit:
        user.save()
    return user

# AgencyRegistrationForm.save()
def save(self, commit=True):
    user = super().save(commit=False)
    user.role = User.Role.AGENCY # Always set to Agency
    if commit:
        user.save()
    return user
```

Custom Validators (core/validators.py)

PYTHON

```
phone_validator = RegexValidator(
    regex=r'^\d{10}$',
    message='Phone number must be exactly 10 digits.',
)

pincode_validator = RegexValidator(
    regex=r'^\d{6}$',
    message='Pin code must be exactly 6 digits.',
)
```

12. Template Inheritance & Frontend Architecture

► MERMAID DIAGRAM

```
graph TD
    subgraph "Template Hierarchy"
        A["base.html<br/>Master Layout<br/>(114 lines)"]
        A --> B["core/landing.html<br/>(15,823 bytes)"]
        A --> C["core/services.html<br/>(5,875 bytes)"]
        A --> D["core/support.html<br/>(7,474 bytes)"]
        A --> E["auth/login.html<br/>(3,566 bytes)"]
        A --> F["auth/register.html<br/>(6,447 bytes)"]
    end

    subgraph "Partials ({% include %})"
        G["core/_navbar.html<br/>(5,979 bytes)"]
        H["core/_footer.html<br/>(6,021 bytes)"]
    end

    A -->|"{% include %}"| G
    A -->|"{% include %}"| H

    subgraph "Frontend Libraries (CDN)"
        I["Tailwind CSS (CDN)"]
        J["HTMX 2.0.4"]
        K["Lucide Icons"]
        L["Custom CSS<br/>(static/css/style.css)"]
    end

    A -->|"<script>"| I
    A -->|"<script>"| J
    A -->|"<script>"| K
    A -->|"<link>"| L

    style A fill:#f59e0b,color:#000
    style B fill:#3b82f6,color:#fff
    style C fill:#3b82f6,color:#fff
    style D fill:#3b82f6,color:#fff
    style E fill:#ef4444,color:#fff
    style F fill:#ef4444,color:#fff
```

Base Template Blocks (templates/base.html)

HTML

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
  <meta name="description" content="{% block meta_description %}...{% endblock %}">
  <title>{% block title %}Helping Hand{% endblock %} – Home Healthcare</title>

  <script src="https://cdn.tailwindcss.com"></script>      <!-- Tailwind -->
  <link rel="stylesheet" href="{% static 'css/style.css' %}"> <!-- Custom CSS -->
  <script src="https://unpkg.com/htmx.org@2.0.4"></script>  <!-- HTMX -->
  <script src="https://unpkg.com/lucide@latest"></script>  <!-- Icons -->

  {% block extra_head %}{% endblock %}
</head>
<body>
  {% include "core/_navbar.html" %}      <!-- Reusable navbar -->

  {% if messages %}
    <!-- Flash messages – auto-dismiss after 5s -->
    {% for message in messages %}
      <div class="card animate-fade-in-up border-l-4 ...">{{ message }}</div>
    {% endfor %}
  {% endif %}

  <main>{% block content %}{% endblock %}</main>

  {% include "core/_footer.html" %}      <!-- Reusable footer -->

  <script>
    // Initialize + re-init icons after HTMX partial swaps
    document.addEventListener('DOMContentLoaded', () => lucide.createIcons());
    document.body.addEventListener('htmx:afterSwap', () => lucide.createIcons());
  </script>

  {% block extra_scripts %}{% endblock %}
</body>
</html>
```

HTMX Integration Pattern

► MERMAID DIAGRAM

```
sequenceDiagram
    participant User
    participant HTMX
    participant Django

    User->>HTMX: Click element with<br/>hx-get or hx-post
    HTMX->>Django: AJAX request<br/>(adds HX-Request header)
    Django->>Django: Render partial template<br/>(or full page)
    Django-->>HTMX: HTML fragment
    HTMX->>HTMX: Swap into hx-target element
    HTMX->>HTMX: Re-initialize Lucide icons

    Note over HTMX: No full page reload!<br/>Smooth, app-like UX
```

Tailwind Configuration (in base.html)

JAVASCRIPT


```
tailwind.config = {
  theme: {
    extend: {
      fontFamily: {
        heading: ['Inter', 'system-ui', 'sans-serif'],
        body: ['DM Sans', 'system-ui', 'sans-serif'],
      },
    },
  },
}
```

13. Django Admin Configuration

The admin interface (core/admin.py) provides a comprehensive back-office management system:

► MERMAID DIAGRAM

```
graph TD
  subgraph "Django Admin (/admin/)"
    A["UserAdmin<br/>(extends BaseUserAdmin)"]
    B["StaffProfileAdmin<br/>(custom ModelAdmin)"]
    C["BookingAdmin<br/>(with ReviewInline)"]
    D["ReviewAdmin<br/>(standalone + inline)"]
  end

  subgraph "UserAdmin Features"
    A --> A1["list_display: username, name,<br/>phone, role, profile_complete"]
    A --> A2["list_filter: role, profile_complete,<br/>is_active, date_joined"]
    A --> A3["Extra fieldsets: Role & Contact,<br/>Address (collapsible), Profile Status"]
  end

  subgraph "BookingAdmin Features"
    C --> C1["list_display: booking_id, patient,<br/>service, status, cost, dates"]
    C --> C2["date_hierarchy: created_at"]
    C --> C3["readonly_fields: all pricing<br/>(auto-computed, can't edit)"]
    C --> C4["ReviewInline: StackedInline<br/>(one review per booking)"]
  end

  subgraph "StaffProfileAdmin Features"
    B --> B1["list_editable: available<br/>(toggle directly from list)"]
    B --> B2["Fieldsets: Personal Info,<br/>Professional, Performance, Ownership"]
  end

  style A fill:#8b5cf6,color:#fff
  style B fill:#8b5cf6,color:#fff
  style C fill:#8b5cf6,color:#fff
  style D fill:#8b5cf6,color:#fff
```

Key Admin Code (core/admin.py)

PYTHON

```
@admin.register(Booking)
class BookingAdmin(admin.ModelAdmin):
    list_display = [
        'booking_id', 'patient_name', 'service_type', 'status',
        'start_date', 'duration_days', 'total_cost', 'staff', 'created_at',
    ]

    # All pricing fields are read-only (auto-computed by model save())
    readonly_fields = [
        'booking_id', 'base_rate', 'discount_percentage',
        'discount_amount', 'subtotal', 'total_cost', 'pricing_tier',
        'created_at', 'updated_at',
    ]

    # Show review inline within booking detail
    inlines = [ReviewInline]

    # Navigate bookings by date
    date_hierarchy = 'created_at'
```

PYTHON

```
@admin.register(StaffProfile)
class StaffProfileAdmin(admin.ModelAdmin):
    list_display = ['name', 'gender', 'age', 'get_service_types',
                   'hourly_rate', 'rating', 'available', 'agency']
    list_editable = ['available'] # Toggle availability from list view!
```

14. Testing Architecture

► MERMAID DIAGRAM

```
graph TD
    subgraph "Test Suite (core/tests/)"
        A["test_models.py<br/>(19,714 bytes)"]
        B["test_auth.py<br/>(14,206 bytes)"]
    end

    subgraph "test_models.py Coverage"
        A --> A1["User Model Tests<br/>role properties, validation,<br/>__str__ representation"]
        A --> A2["StaffProfile Tests<br/>gender choices, service types,<br/>JSONField validation,..."]
        A --> A3["Booking Tests<br/>ID generation (BK-xxx), pricing,<br/>status transitions, busi..."]
        A --> A4["Review Tests<br/>1:1 constraint enforcement,<br/>completed-only rule"]
        A --> A5["Pricing Engine Tests<br/>all tiers, one-time flat rate,<br/>edge cases, format..."]
    end

    subgraph "test_auth.py Coverage"
        B --> B1["Registration Tests<br/>patient + agency sign-up,<br/>auto-role assignment"]
        B --> B2["Login Tests<br/>valid/invalid credentials,<br/>redirect behavior"]
        B --> B3["Logout Tests<br/>session cleanup,<br/>message display"]
        B --> B4["Permission Tests<br/>role-based access control,<br/>403 for wrong role"]
        B --> B5["Form Validation Tests<br/>phone 10-digit, required fields,<br/>duplicate userna..."]
    end

    style A fill:#10b981,color:#fff
    style B fill:#10b981,color:#fff
```

Test Configuration (helpinghand/test_settings.py)

Overrides database to use SQLite for faster in-memory testing:

PYTHON

```
# Run all tests
python manage.py test core.tests --settings=helpinghand.test_settings

# Run specific modules
python manage.py test core.tests.test_models --settings=helpinghand.test_settings
python manage.py test core.tests.test_auth --settings=helpinghand.test_settings
```

15. Deployment & Configuration

Environment Variables Flow

► MERMAID DIAGRAM

```
flowchart LR
    A[".env file"] -->|python-dotenv<br/>load_dotenv()| B["settings.py"]

    subgraph "Security"
        C["DJANGO_SECRET_KEY"]
        D["DJANGO_DEBUG"]
        E["DJANGO_ALLOWED_HOSTS"]
    end

    subgraph "Database"
        F["DB_NAME = helpinghand"]
        G["DB_USER = postgres"]
        H["DB_PASSWORD"]
        I["DB_HOST = localhost"]
        J["DB_PORT = 5432"]
    end

    C --> B
    D --> B
    E --> B
    F --> B
    G --> B
    H --> B
    I --> B
    J --> B

    B --> K["Django Application"]
    K --> L["PostgreSQL"]

    style A fill:#f59e0b,color:#000
    style K fill:#092e20,color:#fff
    style L fill:#336791,color:#fff
```

Key Settings (helpinghand/settings.py)

PYTHON

```
# Security – loaded from .env with safe dev defaults
SECRET_KEY = os.getenv('DJANGO_SECRET_KEY', 'django-insecure-dev-key-...')
DEBUG = os.getenv('DJANGO_DEBUG', 'True').lower() in ('true', '1', 'yes')
ALLOWED_HOSTS = os.getenv('DJANGO_ALLOWED_HOSTS', 'localhost,127.0.0.1').split(',')

# Database – PostgreSQL
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': os.getenv('DB_NAME', 'helpinghand'),
        'USER': os.getenv('DB_USER', 'postgres'),
        'PASSWORD': os.getenv('DB_PASSWORD', 'postgres'),
        'HOST': os.getenv('DB_HOST', 'localhost'),
        'PORT': os.getenv('DB_PORT', '5432'),
        'OPTIONS': {'connect_timeout': 5},
    }
}

# Custom User Model
AUTH_USER_MODEL = 'core.User'

# Timezone (India Standard Time)
TIME_ZONE = 'Asia/Kolkata'

# Static & Media files
STATIC_URL = 'static/'
STATICFILES_DIRS = [BASE_DIR / 'static']
MEDIA_URL = 'media/'
MEDIA_ROOT = BASE_DIR / 'media'
```

16. Complete End-to-End Data Flow

► MERMAID DIAGRAM

```

flowchart TB
    subgraph "Patient Journey"
        P1["1. Visit Landing Page (/)"] --> P2["2. Browse Services (/services/)"]
        P2 --> P3["3. Register as Patient<br/>(/auth/register/?role=patient)"]
        P3 --> P4["4. Complete Profile<br/>(address, alternate phone)"]
        P4 --> P5["5. Create Booking<br/>(service type, dates, patient info)"]
        P5 --> P6["6. Upload Discharge Docs<br/>(optional → auto 'reviewing')"]
        P6 --> P7["7. Booking Auto-Priced<br/>(tiered discount computed)"]
        P7 --> P8["8. Wait for Confirmation<br/>(agency reviews)"]
        P8 --> P9["9. Receive Care<br/>(status: in-progress)"]
        P9 --> P10["10. Submit Review<br/>(1-5 stars, comment)"]
    end

    subgraph "Agency Journey"
        A1["1. Register as Agency<br/>(/auth/register/?role=agency)"] --> A2["2. Add Staff Profil..."]
        A2 --> A3["3. Set Staff Availability<br/>& Service Types"]
        A3 --> A4["4. Review Incoming Bookings<br/>(status: pending/reviewing)"]
        A4 --> A5["5. Accept & Assign Staff<br/>(gender + service matching)"]
        A5 --> A6["6. Monitor In-Progress<br/>Bookings"]
        A6 --> A7["7. Mark Complete<br/>(triggers review eligibility)"]
    end

    subgraph "System Automations (model save)"
        S1["Auto-generate Booking ID<br/>(BK-xxxxxxx via UUID)"]
        S2["Auto-compute Pricing<br/>(tiered discounts on every save)"]
        S3["Auto-set Status<br/>(discharge file → reviewing)"]
        S4["Gender Matching Validation<br/>(staff.gender == patient_gender)"]
        S5["Service Type Validation<br/>(service_type in staff.service_types)"]
        S6["Review Constraint<br/>(only for completed bookings, 1:1)"]
    end

    P5 --> S1
    P5 --> S2
    P6 --> S3
    A5 --> S4
    A5 --> S5
    P10 --> S6

    style P1 fill:#3b82f6,color:#fff
    style A1 fill:#10b981,color:#fff
    style S1 fill:#f59e0b,color:#000
    style S2 fill:#f59e0b,color:#000
    ... (truncated)

```

Quick Start

BASH

```
# 1. Install dependencies
pip install -r requirements.txt

# 2. Create PostgreSQL database
createdb helpinghand

# 3. Configure environment
cp .env.example .env # Edit DB credentials

# 4. Run migrations
python manage.py migrate

# 5. Create superuser for admin access
python manage.py createsuperuser

# 6. Start development server
python manage.py runserver

# 7. Access the application
# Landing page: http://localhost:8000/
# Admin panel: http://localhost:8000/admin/
# Register: http://localhost:8000/auth/register/?role=patient
# Login: http://localhost:8000/auth/login/
# Services: http://localhost:8000/services/
# Support/FAQ: http://localhost:8000/support/
```

Architecture documentation for the Helping Hand MAX codebase — a Django healthcare marketplace migrated from React/Firebase.

Generated: May 2026